INDIAN INSTITUTE OF TECHNOLOGY KANPUR

Department of Computer Science and Engineering

CS666 – Hardware Security for IoT

Semester-I, Academic Year 2025–26

Project Report
Pipelined AES-128 Encryption Module on PYNQ Board

Instructor: Prof. Urbi Chatterjee

Group Number: 8

Group Members:

1. Khushal Nikam – 251110043
2. Pranjal – 251110058
3. Sarthak Dumbre – 251110064
4. Vishal Junjare – 251110085

Index

Contents

1 Abstract

This report presents the design and implementation of a pipelined AES-128 encryption module for the PYNQ board. The design focuses on achieving high throughput and low latency using a sequential key expansion architecture.

2 Introduction

As we know, AES is one of the most widely used symmetric key encryption algorithms. It is a block cipher with a block size of 128 bits and supports key sizes of 128, 192, and 256 bits. Depending on the key size, it consists of 10 (128-bit key size), 12 (192-bit key size) and 14 rounds (256-bit key size). Each round, except the last round (which excludes the Mix Column), consists of four operations: SubBytes, ShiftRow, MixColumns, and Add Round Key.

Although software implementation of AES is present, hardware-based implementations are preferred when high throughput and low latency are required. Different types of hardware-based implementations can be done for AES, including Rolled, Unrolled, and Pipelined implementations. Each of these designs has its own trade-offs with respect to area, latency, and speed.

2.1 Rolled Architecture

In this implementation design, only one round is implemented in hardware, and it is reused for all the rounds. A counter is maintained to keep track of the round being executed. This approach needs less area (as only one round hardware is used), but the throughput is low (as each block takes multiple cycles to encrypt). This design is useful for area-constrained systems where performance is not the main focus.

2.2 Unrolled Architecture

This design is another extreme of hardware implementation, where all the rounds are implemented in parallel in hardware. Here, every input block passes through all the rounds in one clock cycle. Although the throughput is high, this approach requires large area usage and a longer critical path. These architectures are generally used when performance is the main focus and area and power consumption are secondary factors.

2.3 Pipelined Architecture

The pipelined architecture lies between the above two extremes. Here, the rounds of AES are split into multiple pipeline stages, where each stage performs a part of the encryption

process. Once the pipeline is filled (i.e., after initial latency), the design outputs one encrypted block per clock cycle.

This design achieves high throughput similar to the unrolled version but with lower area usage since the encryption logic is shared across pipeline stages. However, this design has the main drawback of initial latency because the first output can only be available after the pipeline is filled; after that, the encryption output is continuous, i.e., one block every cycle.

It is because of the balance between area usage and performance that the pipelined architecture is widely used in FPGA-based systems.

3 Design Architecture

3.1 Overview of Design

The design architecture adopts a **half-round pipeline** approach aimed at achieving high throughput and optimized hardware efficiency. The architecture divides the encryption process into multiple pipeline stages, each corresponding to an AES round, allowing concurrent processing of data blocks. To further increase throughput we have designed internal pipelining within each round which minimizes critical path delay.

- Each AES round is divided into two stages:
  1. **Stage A:** SubBytes + ShiftRows
  2. **Stage B:** MixColumns + AddRoundKey
- The pipeline allows one new 128-bit plaintext to be accepted every clock cycle.
- A parallel key generation unit producing round keys `rk0--rk10`.
- Used registers as buffer for storing information calculated in current stage to be used in next stage.

3.2 Initial input processing

Before the first round, the 128-bit plaintext data block undergoes an initial `AddRoundKey` transformation. As shown in the block diagram, this stage consists of a bitwise XOR operation between the plaintext and the initial key.

3.3 Round Pipelining (Rounds 1-9)

Following the initial transformation, the data enters the main pipeline, which consists of 10 rounds. As you've noted, each of the first nine rounds is divided into two distinct pipeline stages to shorten the critical path:

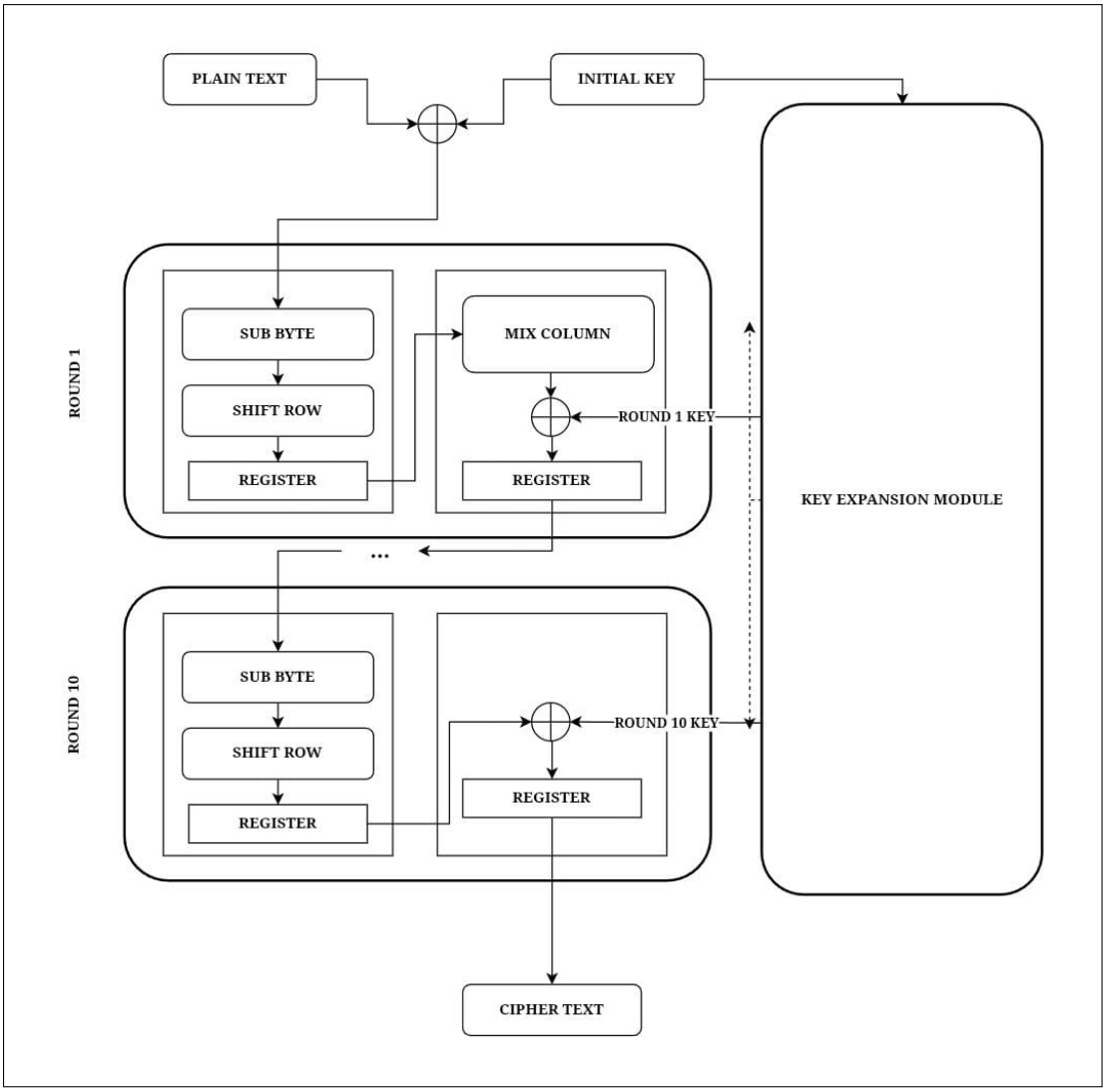

Figure 1: Half round AES pileline architecture

- **Stage A:** Performs the `SubBytes` and `ShiftRows` transformations. The `SubBytes` operation is a non-linear byte substitution using a lookup table (S-box), and `ShiftRows` is a simple byte-wise permutation.
- **Stage B:** Performs the `MixColumns` and `AddRoundKey` transformations. `MixColumns` provides diffusion through a matrix multiplication in $GF(2^8)$, and `AddRoundKey` XORs the state with the corresponding round key (`rk1` to `rk9`).

The output of Stage A is latched into a pipeline register before being fed to Stage B, and the output of Stage B is latched before being fed to Stage A of the next round.

3.4 Final Round Exception (Round 10)

As per the AES specification, the final round (Round 10) is a variation of the preceding rounds, as it **omits the `MixColumns` operation**. Our design correctly implements this

exception. As illustrated in the diagram, Round 10 is still divided into two pipeline stages to maintain timing consistency:

- **Stage 10A:** Performs `SubBytes` and `ShiftRows`.
- **Stage 10B:** Performs only the final `AddRoundKey` operation, XORing the state with `rk10`.

The output of Stage 10B is the final 128-bit ciphertext.

3.5 Key Expansion Module

The AES key expansion is implemented as a separate sequential pipeline (`KeyPipe`) running parallel to the data path. The `KeyPipe` module generates all ten round keys (`rk1--rk10`) using the AES key schedule algorithm.

Each clock cycle, a new round key is derived from the previous one using the `Key` module. The generated round keys are stored in internal registers (`k1--k10`) so that each encryption round receives the correct round key at the right time. This sequential approach avoids the large combinational delay associated with a fully combinational key expansion circuit.

This implementation (*refer to* : 6) works even if the Key changes per plaintext. Since key expansion is not our critical path, further improving its performance will not affect our clock period.

3.6 Data Flow and Pipeline Registers

The entire design functions as a 20-stage pipeline: 18 stages for the 9 core rounds (2 stages/round), and 2 stages for the final 10th round.

Pipeline registers are used as buffers between every stage. At each positive clock edge, the combinational logic of a stage completes, and its output is latched by a register. This registered output becomes the stable input for the next pipeline stage during the following clock cycle.

4 Implementation Details

4.1 AES_Encryption Main Module

```
// ============================================================================
// Top-level AES Encryption (half-round pipeline)
// ============================================================================
module AES_Encryption(
    input              clk,
    input      [127:0] Data_in,
    input      [127:0] key_in,
    output     [127:0] cipher_out
);

    // -------- Input registers --------
    reg [127:0] din_reg, key_reg;
    always @(posedge clk) begin
        din_reg <= Data_in;
        key_reg <= key_in;
    end

    // -------- Key generation --------
    wire [127:0] rk0, rk1, rk2, rk3, rk4, rk5, rk6, rk7, rk8, rk9, rk10;
    KeyPipe KP (
        .clk(clk),
        .key_in(key_reg),
        .rk0(rk0), .rk1(rk1), .rk2(rk2), .rk3(rk3), .rk4(rk4),
        .rk5(rk5), .rk6(rk6), .rk7(rk7), .rk8(rk8), .rk9(rk9), .rk10(rk10)
    );
```

Figure 2: Top-level AES_Encryption Module

The above code snippet describes our Top Level module (**AES Encryption**). This module integrates the complete encryption datapath and key expansion pipeline. As we can see, inputs for this module are a 128-bit plaintext block (`data_in`), a 128-bit secret key (`key_in`) and the system clock (`clk`).

This module first stores the input in the internal register to synchronize it with the pipeline. After that, it uses the `KeyPipe` module to sequentially generate all 10 round keys (`rk0` to `rk10`).

```
// -------- Initial AddRoundKey --------
wire [127:0] s0 = din_reg ^ rk0;

// -------- Round 1 --------
wire [127:0] r1a, r1b;
round_half_a R1A (.clk(clk), .din(s0), .q(r1a));
round_half_b #(.USE_MC(1)) R1B (.clk(clk), .din(r1a), .rk(rk1), .q(r1b));

// -------- Round 2 --------
wire [127:0] r2a, r2b;
round_half_a R2A (.clk(clk), .din(r1b), .q(r2a));
round_half_b #(.USE_MC(1)) R2B (.clk(clk), .din(r2a), .rk(rk2), .q(r2b));

// -------- Round 3 --------
wire [127:0] r3a, r3b;
round_half_a R3A (.clk(clk), .din(r2b), .q(r3a));
round_half_b #(.USE_MC(1)) R3B (.clk(clk), .din(r3a), .rk(rk3), .q(r3b));

// -------- Round 4 --------
wire [127:0] r4a, r4b;
round_half_a R4A (.clk(clk), .din(r3b), .q(r4a));
round_half_b #(.USE_MC(1)) R4B (.clk(clk), .din(r4a), .rk(rk4), .q(r4b));

// -------- Round 5 --------
wire [127:0] r5a, r5b;
round_half_a R5A (.clk(clk), .din(r4b), .q(r5a));
round_half_b #(.USE_MC(1)) R5B (.clk(clk), .din(r5a), .rk(rk5), .q(r5b));

// -------- Round 6 --------
wire [127:0] r6a, r6b;
round_half_a R6A (.clk(clk), .din(r5b), .q(r6a));
round_half_b #(.USE_MC(1)) R6B (.clk(clk), .din(r6a), .rk(rk6), .q(r6b));

// -------- Round 7 --------
wire [127:0] r7a, r7b;
round_half_a R7A (.clk(clk), .din(r6b), .q(r7a));
round_half_b #(.USE_MC(1)) R7B (.clk(clk), .din(r7a), .rk(rk7), .q(r7b));

// -------- Round 8 --------
wire [127:0] r8a, r8b;
round_half_a R8A (.clk(clk), .din(r7b), .q(r8a));
round_half_b #(.USE_MC(1)) R8B (.clk(clk), .din(r8a), .rk(rk8), .q(r8b));

// -------- Round 9 --------
wire [127:0] r9a, r9b;
round_half_a R9A (.clk(clk), .din(r8b), .q(r9a));
round_half_b #(.USE_MC(1)) R9B (.clk(clk), .din(r9a), .rk(rk9), .q(r9b));

// -------- Final Round (NO MixColumns) --------
wire [127:0] r10a, r10b;
round_half_a R10A (.clk(clk), .din(r9b), .q(r10a));
round_half_b #(.USE_MC(0)) R10B (.clk(clk), .din(r10a), .rk(rk10), .q(r10b));

assign cipher_out = r10b;
```

Figure 3: AES Core Encryption Pipeline

This section of the module forms the core encryption pipeline by defining the data flow through each of the AES rounds. In our pipelined design, after the `AddRoundKey` step, ten rounds are executed sequentially. Each round is divided into two stages for processing (two-stage pipelining):

- **Half-Round A** performs the `SubBytes` and `ShiftRows` operations.
- **Half-Round B** performs the `MixColumns` and `AddRoundKey` transformations.

Each Half round includes the pipeline register, which ensures that the design can

process one block per clock cycle once the pipeline is filled. The signals (`r1a, r1b, ... r9a, r9b`) represent the intermediate states and are outputs of each substage.

```
// ============================================================================
// Half-round A  : SubBytes + ShiftRows + pipeline reg
// ============================================================================
module round_half_a(
    input  clk,
    input  [127:0] din,
    output [127:0] q
);
    wire [127:0] sb, sr;
    Sub_Bytes  SB (.A(din), .B1(sb));
    Shift_Rows SR (.b_in(sb), .b_out(sr));
    Round_reg  RR (.clk(clk), .r_in(sr), .r_out(q));
endmodule
```

Figure 4: Half-Round A Implementation

The above code snippet describes the implementation of **Half-Round A** submodule. As we can see, it performs the `SubBytes` and `ShiftRows` transformations on the input state. Then we store the output of the `ShiftRows` operation into the pipeline register using the `Round_reg` module, which eventually synchronises it with the clock.

```
// ============================================================================
// Half-round B : MixColumns (Optional) + AddRoundKey + pipeline reg
// ============================================================================
module round_half_b #(
    parameter USE_MC = 1
) (
    input  clk,
    input  [127:0] din,
    input  [127:0] rk,
    output [127:0] q
);
    wire [127:0] temp, ark;
    generate
        if (USE_MC) begin : WITH_MC
            Mix_Column MC (.c_in1(din), .c_out(temp));
        end else begin : NO_MC
            assign temp = din;
        end
    endgenerate

    Sub_Key   AK (.key_in(rk), .mc_sr_out(temp), .data_out(ark));
    Round_reg RR (.clk(clk), .r_in(ark), .r_out(q));
endmodule
```

Figure 5: Half-Round B Implementation

The above code snippet describes the implementation of the **Half-Round B** submodule. It performs two main operations: `MixColumns` and `AddRoundKey`.

Here, to take care of the last round, the `MixColumns` transformation is made optional and controlled by the `USE_MC` parameter. For Rounds 1–9, `USE_MC = 1`, and for Round 10, `USE_MC = 0` (so that the `MixColumns` operation is omitted).

The `Sub_Key` submodule is responsible for performing the `AddRoundKey` operation. Finally, the `Round_reg` module stores the output of this half-round for the next pipeline stage.

```
// ============================================================================
// KeyPipe
// ============================================================================
module KeyPipe(
  input             clk,
  input      [127:0] key_in,
  output     [127:0] rk0, rk1, rk2, rk3, rk4, rk5, rk6, rk7, rk8, rk9, rk10
);
  reg [127:0] k0, k1, k2, k3, k4, k5, k6, k7, k8, k9, k10;
  wire [127:0] k1_n, k2_n, k3_n, k4_n, k5_n, k6_n, k7_n, k8_n, k9_n, k10_n;

  Key K1 (.rc(4'd0), .k_in(k0),  .k_out(k1_n));
  Key K2 (.rc(4'd1), .k_in(k1),  .k_out(k2_n));
  Key K3 (.rc(4'd2), .k_in(k2),  .k_out(k3_n));
  Key K4 (.rc(4'd3), .k_in(k3),  .k_out(k4_n));
  Key K5 (.rc(4'd4), .k_in(k4),  .k_out(k5_n));
  Key K6 (.rc(4'd5), .k_in(k5),  .k_out(k6_n));
  Key K7 (.rc(4'd6), .k_in(k6),  .k_out(k7_n));
  Key K8 (.rc(4'd7), .k_in(k7),  .k_out(k8_n));
  Key K9 (.rc(4'd8), .k_in(k8),  .k_out(k9_n));
  Key KA (.rc(4'd9), .k_in(k9),  .k_out(k10_n));

  always @(posedge clk) begin
    k0  <= key_in;
    k1  <= k1_n;
    k2  <= k2_n;
    k3  <= k3_n;
    k4  <= k4_n;
    k5  <= k5_n;
    k6  <= k6_n;
    k7  <= k7_n;
    k8  <= k8_n;
    k9  <= k9_n;
    k10 <= k10_n;
  end

  assign rk0=k0;  assign rk1=k1;  assign rk2=k2;  assign rk3=k3;  assign rk4=k4;
  assign rk5=k5;  assign rk6=k6;  assign rk7=k7;  assign rk8=k8;  assign rk9=k9;
  assign rk10=k10;
endmodule
```

Figure 6: KeyPipe Module Implementation

The `KeyPipe` module implements the AES key expansion process. It accepts the initial 128-bit input key (`key_in`) and produces a sequence of keys (`rk0` to `rk10`) corresponding to each AES round.

Internally, it uses multiple instances of the `Key` submodule, each responsible for deriving the next round key based on the previous one. The `Key` submodule follows the AES key schedule rules, such as byte rotation, substitution, and Rcon addition.

All round keys are then updated and stored in registers on each rising clock edge, ensuring proper synchronisation with the encryption pipeline.

4.2 Imported Modules

The following external modules are used in the `AES_Encryption` module, and their functions are briefly described below:

- **Key.v** – It generates and expands the AES master key into round keys used in each encryption round. It ensures that every round gets the correct key at the right time.
- **Mix_Column.v** – It performs the MixColumns operation, that is, it mixes bytes in each column to spread data according to the standard MixColumn procedure used in AES algorithms.
- **Round_reg.v** – It stores intermediate data between rounds and keeps the timing and data flow synchronised in the encryption process. It serves as the pipeline register for the design.
- **Sbox.v** – It is the AES S-Box, which is a lookup table used to replace bytes during the Substitute-Byte operation. This module is called as a subroutine in the SubBytes module.
- **Shift_Rows.v** – It shifts the rows of the AES state matrix by fixed amounts (no shift for Row 0, left shift by 1 for Row 1, left shift by 2 for Row 2, and left shift by 3 for Row 3).
- **Sub_Bytes.v** – It replaces each byte in the AES state using values from the S-Box and therefore adds nonlinearity to each round.
- **Sub_Key.v** – It performs the AddRoundKey step by performing the XOR operation between the round key and the state matrix data.

5 Optimization Techniques

The proposed **Half-Round Pipelined Architecture** has several key optimizations over the original non-pipelined AES implementation given in the original *HelloIITK* design. Each modification specifically targets improvements in **throughput**, **clock frequency**, and **hardware efficiency**, also making sure that it meets all the constraints.

5.1 Introduction of Pipelining

The original AES design was entirely **combinational**, which executes all the rounds one by one without any pipeline registers. Which results in a long critical path through multiple stages, and thus we get a lower clock frequency.

In the proposed design, the AES encryption process is divided into **half-round pipeline stages**:

- **Stage 1 (Half-Round A):** SubBytes + ShiftRows
- **Stage 2 (Half-Round B):** MixColumns + AddRoundKey

Pipeline registers are inserted between each half-round, which allows every stage to get completed in one clock cycle. Then, after an initial latency of **20 clock cycles**, a new ciphertext is produced every clock cycle, having a throughput of one block per cycle.

5.2 Sequential Key Expansion (KeyPipe)

In the non-pipelined AES, the entire key expansion was happening in one single stage, All the 10 round keys were generated **combinationally** in a single step,which was the critical path of the Encryption.

The optimized design uses a **sequential KeyPipe module**, which performs key expansion in multiple clock cycles.In each cycle new round key is generated and stored in a pipeline register. The key generation process is in sync with the Encryption data flow, which makes sure that every round key is available when it is needed. This makes sure that Key Expansion is no longer our critical path.

Methods to reduce critical path delay and area utilization.

5.3 Balanced Pipeline Depth and Area

Overall, the half-round pipelined AES-128 encryption design achieves high throughput, improved clock frequency, without increasing a lot of hardware. It shows the advantages of pipelining and sequential key expansion in AES Encryption.

- **Latency:** 20 clock cycles
- **Throughput:** One 128-bit ciphertext per clock cycle after first ciphertext
- **Area:** Slightly higher than non-pipelined AES.

This balanced pipeline reduces the critical path delay compared to the original non-pipelined AES design. As a result, the design achieves a much higher clock frequency.

6 Simulation and Verification

6.1 Functional Verification

We wrote a simple testbench that gives multiple 128-bit plaintext blocks and a 128-bit initial key to the AES pipeline. The testbench checks each output block against known AES-128 test vectors which we already know. The testbench also prints timing markers (input accepted cycle, output produced cycle) for each block so that the latency and the actual throughput we can measure to see the performance of our code.

6.2 Verification of Correctness

- **Reference Vectors:** Used AES test vectors (plaintext → ciphertext) to check whether we are getting correct and expected output.
- **Output Comparison:** Each ciphertext output from the Verilog module was compared bit-by-bit with the corresponding expected output which we already had. The simulation console prints "PASS (matches expected)" whenever the output matches.

Figure 7: Final Output after Execution

6.3 Timing Diagrams

The simulation waveform was captured for the first encryption transaction to visualize how the pipeline is running and checked how it is producing the first output and also the next corresponding outputs. The key signals displayed in the waveform are:

- `clk` – clock signal
- `in_valid` – indicates when input data is valid
- `Data_in` – input plaintext data
- `out_valid` – indicates when output data is valid
- `cipher_out` – output ciphertext data

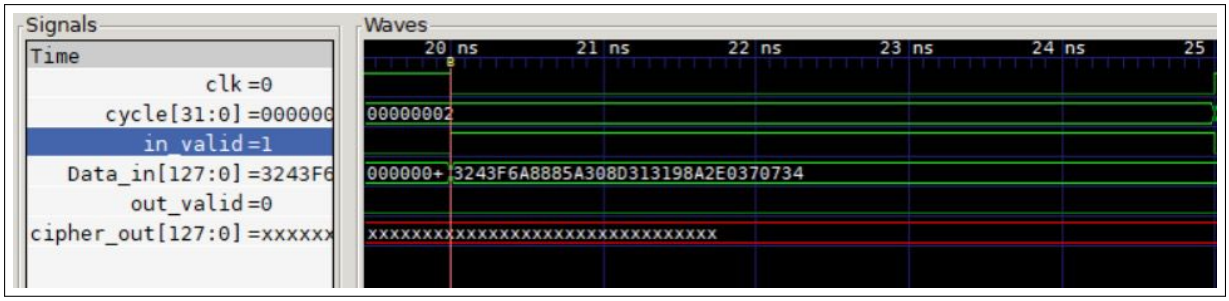

Figure 8: Input time of initial Plaintext

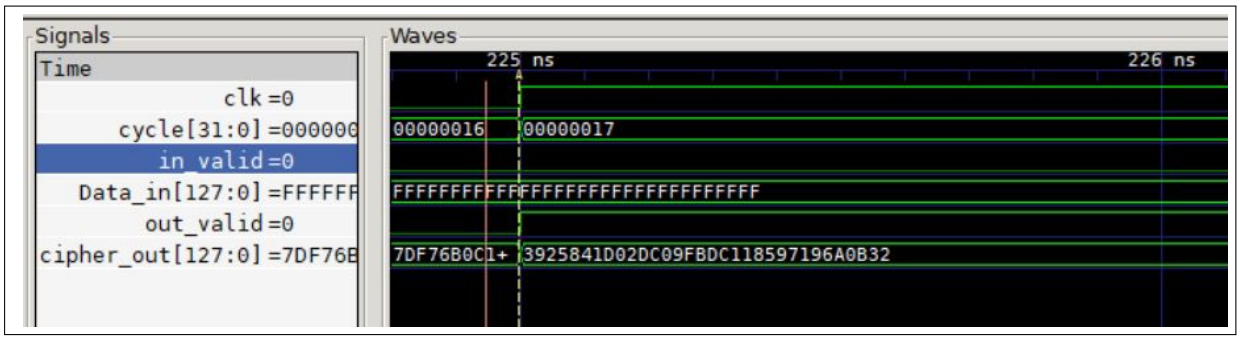

Figure 9: Waveform showing the first valid ciphertext output after the initial latency

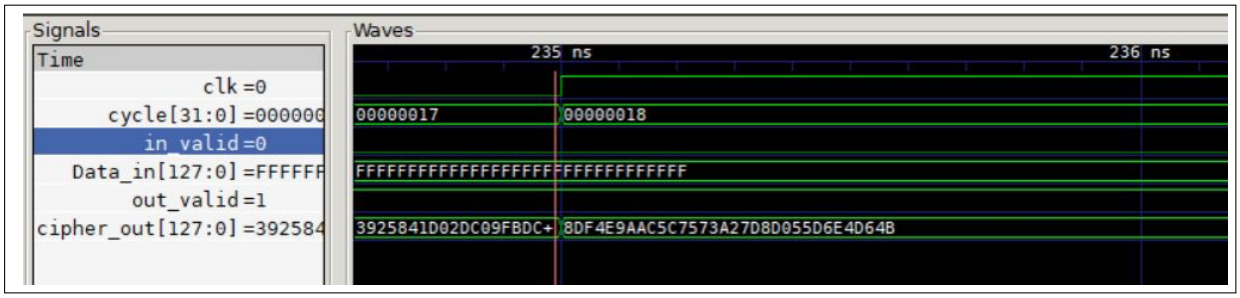

Figure 10: Waveform showing the second valid ciphertext output

Note: Cycle time is set to 10 ns for simulation purpose

The first assertion of `out_valid` occurs after the initial latency, as discussed in the results section.

- The first input was fed at 20ns and we got first output around 225ns which shows it is taking **20 cycles**.
- In 9 The output of the first plaintext we are getting at 225ns (The 5ns delay is because the input fed at negative edge and we are getting output at positive edge) We can see clearly from waveform annotations the number of cycles the first encryption took before steady-state operation was reached, after that one ciphertext is produced every clock cycle.

7 Performance Evaluation

In this section we have discussed the overall performance analysis of our Pipelined AES design Implemented on the PYNQ board. The main things we have checked for analysing the performance are- clock frequency, throughput, latency, and FPGA resource utilization obtained from Vivado post-implementation reports. Additionally, the *Throughput* × *Area* metric has been used to evaluate the efficiency of the design.

7.1 Clock and Frequency Analysis

From the Vivado timing summary (see Figure 12), the implemented design achieved a clock period of **4.666 ns**, which corresponds to an operating frequency of approximately **214.316 MHz**. This frequency represents the maximum stable clock rate at which the design can operate without violating timing constraints. The positive Worst Negative Slack (WNS) of **0.212 ns** indicates that the design comfortably meets the timing requirements, and all specified timing constraints are satisfied.

- **Clock period:** 4.666 ns
- **Achieved frequency:** 214.316 MHz
- **Worst Negative Slack (WNS):** +0.212 ns
- **Total Negative Slack (TNS):** 0 ns

7.2 Throughput Calculation

$$\text{Throughput} = (\text{Block Size}) \times (\text{Clock Frequency})$$

We are having:

$$\text{Block Size} = 128 \text{ bits}, \quad \text{Clock Frequency} = 214.316 \text{ MHz}$$

$$\Rightarrow \text{Throughput} = 128 \times 214.316 \times 10^6 = 27.43 \times 10^9 \text{ bits/second}$$

Therefore, the steady-state throughput of the design is:

$$\boxed{\text{Throughput} = 27.43 \text{ Gbps}}$$

This means that once the pipeline is filled, the hardware AES encryption engine can process approximately **27.43 Gega bits per second** in steady-state operation.

7.3 Initial Latency

The latency is defined as the number of cycles between the first input and the corresponding first output block. From the AES testbench console output, it is observed that the first output appears after **20 cycles**. Since the clock period is 4.666 ns, the total latency in time units can be calculated as:

$$\text{Latency (ns)} = 20 \times 4.666 = 93.32 \text{ ns}$$

Hence, the initial latency before the pipeline reaches steady-state operation is approximately:

$$\boxed{\text{Latency} = 20 \text{ cycles } = 93.32 \text{ ns}}$$

7.4 Throughput–Area Product

To evaluate the hardware efficiency of the implemented AES-128 design, the *Throughput–Area Product* metric was calculated. In this evaluation, the area is represented by the number of Look-Up Tables (LUTs) utilized in the FPGA implementation.

$$\text{Throughput–Area Product} = \text{Throughput} \times \text{Number of LUTs}$$

Substituting the obtained values:

$$27.43 \times 10^9 \text{ (bits/s)} \times 10{,}577 \text{ (LUTs)} \approx 2.90 \times 10^{14} \text{ bit·LUT/s}$$

Therefore, the efficiency metric of the design is approximately:

$$\boxed{\text{Throughput–Area Product} \approx 2.90 \times 10^{14} \text{ bit·LUT/s}}$$

Using this LUT-based area estimation provides a close and practical approximation for comparing hardware efficiency.

7.5 FPGA Resource Utilization

Table 1 lists the major hardware resources consumed by the AES-128 pipeline as reported by Vivado after implementation.

These values of resource utilization show that the design occupies less than 20% of the available LUTs, which shows a good balance between area and speed. The moderate power consumption of 1.776 W at 214 MHz also confirms that the pipeline design is power-efficient.

Table 1: FPGA Resource Utilization Summary

Resource Type	**Utilization**
LUTs (Logic)	10,577
LUTRAM	60
Flip-Flops (FF)	5,081
BUFG	1
Total On-chip Power	1.776 W
Junction Temperature	45.5 °C

8 Results

With our implementation, the pipelined AES-128 encryption module was successfully synthesized and tested on the PYNQ board. The final design met all timing requirements and achieved stable performance in both simulation and hardware implementation.

The design produces one 128-bit encrypted block per clock cycle once the pipeline is filled. From the routed timing report, the operating frequency achieved is approximately **214.316 MHz**.

The initial latency, defined as the delay between applying the first input and receiving the first output, was observed to be **20 cycles**, which corresponds to approximately **93.32 ns**.

8.1 Area and Resource Utilization

The post-place and route utilization report from Vivado shows that the design uses **10,577 LUTs**, **5,081 Flip-Flops**, and **1 BUFG**. The design achieved timing closure with a positive Worst Negative Slack (WNS) of **0.212 ns**, indicating no timing violations. The total on-chip power consumption was estimated to be around **1.776 W** at an operating temperature of **45.5 °C**.

Table 2: Summary of Post-Implementation Results

Parameter	**Measured Value**
Operating Frequency	214.316 MHz
Throughput	27.43 Gbps
Initial Latency	20 cycles (93.32 ns)
LUTs Used	10,577
Flip-Flops Used	5,081
BUFG Used	1
Power Consumption	1.776 W
Temperature	45.5 °C
WNS	+0.212 ns

8.2 Screenshots and Evidence

The following figures summarize the results obtained from simulation and synthesis:

- **Figure 11:** Vivado utilization summary displaying LUT, FF, and BUFG counts.
- **Figure 12:** Timing summary indicating the achieved clock frequency and positive slack.

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs	10577	0	0	53200	19.88
LUT as Logic	10517	0	0	53200	19.77
LUT as Memory	60	0	0	17400	0.34
LUT as Distributed RAM	0	0			
LUT as Shift Register	60	0			
Slice Registers	5081	0	0	106400	4.78
Register as Flip Flop	5081	0	0	106400	4.78
Register as Latch	0	0	0	106400	0.00
F7 Muxes	3008	0	0	26600	11.31
F8 Muxes	1184	0	0	13300	8.90

* Warning! LUT value is adjusted to account for LUT combining.

Figure 11: Vivado utilization report showing LUT, FF, and BUFG usage.

Design Timing Summary

WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)
0.212	0.000	0	6437	0.037

All user specified timing constraints are met.

Clock Summary

Clock	Waveform(ns)	Period(ns)	Frequency(MHz)
clk_fpga_0	{0.000 2.333}	4.666	214.316

Figure 12: Vivado routed timing summary indicating achieved frequency (214.316 MHz) and positive slack.

9 Conclusion

In this Assignment, we implemented a pipelined version of AES-128 encryption module on the PYNQ FPGA board. The primary goal was to improve throughput while keeping the design area and latency within a reasonable range. By making the key expansion sequential and pipelining the round operations, the design achieved stable timing and produced one encrypted block per clock cycle after the initial fill.

From the results, the design operates at approximately **214 MHz** and achieves throughput **27.4 Gbps**, which is quite good for a lightweight FPGA setup. The initial latency of 20 cycles is acceptable since, after that, the pipeline continuously outputs one encrypted block per clock. The resource utilization was also good which is around **10.5K LUTs** - showing that the design is efficient both in speed and area.

Overall, this project gave us a solid understanding of how pipelining can drastically improve performance in hardware encryption systems and can make the process fast. We also gained practical experience in synthesis, timing analysis, and optimization using Vivado. With this knowledge we will try to make the other algorithms pipelined implementation to improve their performance in the future.

10 Bibliography

References

[1] Y. Zhang and X. Wang, "Pipelined Implementation of AES Encryption Based on FPGA," *2010 IEEE International Conference on Information Theory and Information Security (ICITIS)*, Beijing, China, pp. 170–173, 2010, doi: 10.1109/ICITIS.2010.5688757.

[2] CS666: Hardware Security for IoT, Indian Institute of Technology Kanpur (IITK), Lecture Notes, 2025.

Disclaimer

Some part of the codebase was generated with the help of AI. All simulation, testing, and report writing were performed by the authors.